Task 1: Design

1. Workflow Design

I will implement a **schema-first extraction pipeline** where the LLM acts as a text understanding component wrapped in deterministic guardrails. The input is messy, informal construction text; the output is strict JSON that must match an agreed schema. In this system, **incorrect or hallucinated data is considered more harmful than missing data**.

Prompting Strategy (System Role + Examples):
A system prompt defines the task, the exact JSON schema, and strict rules such as "do not guess missing values" and "return JSON only." A small number of examples may be included to demonstrate how noisy site messages map to clean structured output, especially for construction-specific language.

Deterministic Configuration:
The model is called with low temperature so it behaves like a consistent parser rather than a creative writer. Identical inputs should yield identical outputs.

Schema Enforcement:
The LLM output is validated against a predefined schema using a structured output mechanism (for example, JSON Schema or a typed validation layer such as Pydantic). Outputs with incorrect types, missing required keys, or extra fields are rejected before they reach downstream systems.

2. Handling Failures and Anomalies

The design assumes the LLM will fail occasionally and handles those failures explicitly.

Missing Fields & Hallucinations:
The prompt explicitly forbids guessing. If `project_name`, `location`, or `deadline` are not clearly present, the LLM must return `null`. During validation, any missing field is set to `null`, and any extra or hallucinated key is removed to preserve schema consistency.

Urgency & Date Inference (Hybrid Logic):
The LLM is limited to extracting raw signals such as "urgent," "ASAP," or a date phrase. Final urgency (`low`, `medium`, `high`) is computed deterministically in Python using explicit rules and, when available, the time difference between a provided `current_date` and the extracted deadline. This avoids subjective or inconsistent urgency decisions by the model.

Context Misinterpretation:
Strict field types and prompt guidance reduce ambiguity. For example, `quantity` must be numeric; if the model outputs text, validation fails. Prompt instructions clarify that names like "Project Phoenix" or "Project Avenue" map to `project_name`, while city or area names map to `location`.

3. Resilience: Auto-Correction Loop

If the LLM produces malformed JSON or fails schema validation, the system does not crash. Instead, it executes a single, controlled retry:
1. Catch the parsing or validation error.
2. Re-prompt the model with the previous output and a brief instruction to fix the error and return valid JSON only.
3. Accept the corrected output if it passes validation; otherwise, log the failure and fall back to a safe default where fields are `null`.

Conclusion

This design treats the JSON schema as a **data contract** and the LLM as a helper operating inside that contract. By combining strict schema enforcement, deterministic post-processing, and bounded error recovery, the system produces reliable, non-hallucinated structured outputs that downstream procurement and billing systems can safely depend on.